

WEB VAPT EVIDENCE FILE DOCUMENTATION

09-dvwa-sqli-vulnerable-input-response.png

Field	Details
Project	Enterprise Security Assessment Lab
Module	Sub-Project 1 - Web Application VAPT
Evidence Row	14 - SQL Injection
Evidence File	09-dvwa-sqli-vulnerable-input-response.png
Result Type	Manual SQLi validation
Prepared For	GitHub portfolio documentation

Document Purpose

This PDF documents one specific evidence file from the Web VAPT evidence table. It embeds the actual screenshot, explains what the screenshot proves, maps it to the methodology, and records professional interpretation, risk, remediation, and redaction guidance for GitHub publication.

1. Embedded Evidence Screenshot

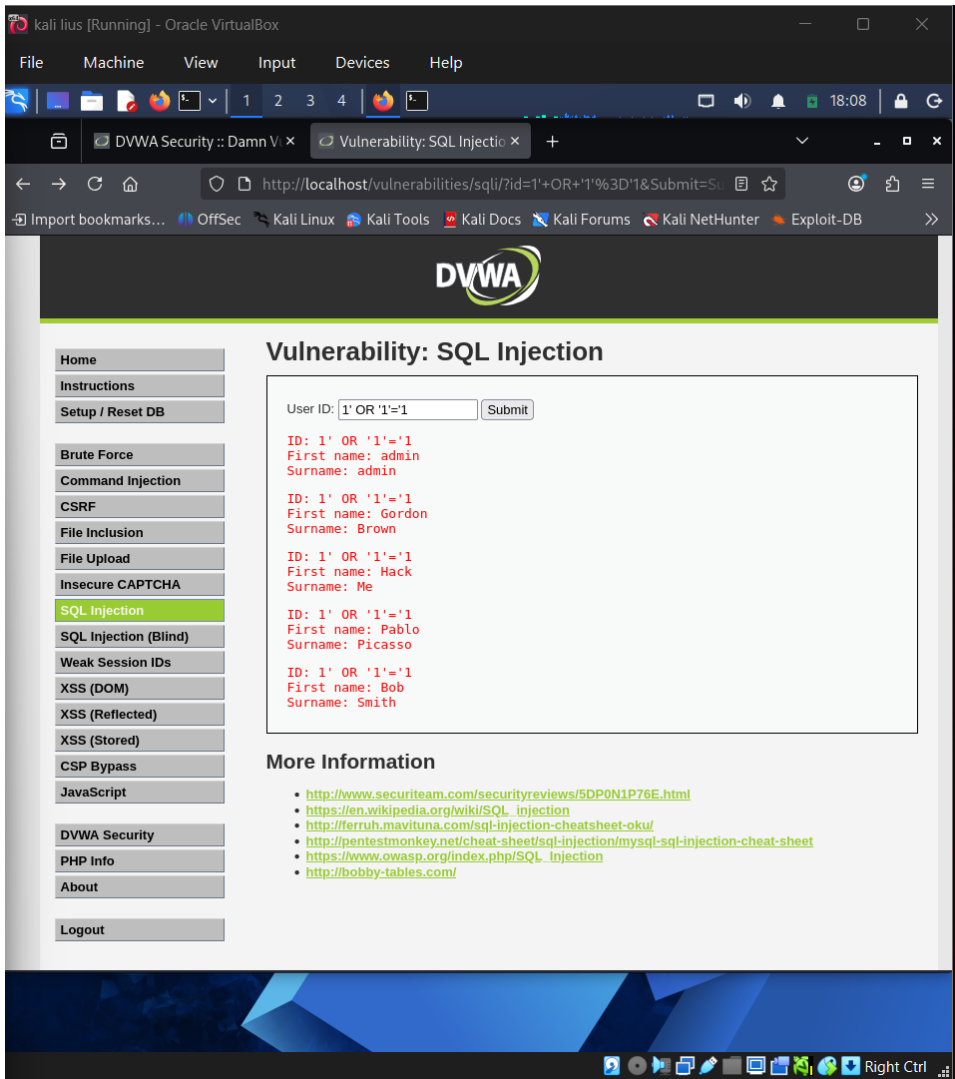


Figure 1 - Actual screenshot evidence: 09-dvwa-sqli-vulnerable-input-response.png

What this screenshot proves

The screenshot shows the injected payload returning multiple user records, demonstrating SQL Injection behaviour in the lab.

2. Evidence Context

Area	Value
Testing Phase	SQL Injection Manual Validation
Tool / Component	Browser / DVWA
Objective	Validate SQL Injection manually before relying on automated tooling.
Result Type	Manual SQLi validation

3. Command / Workflow Used

Payload: 1' OR '1'='1
Submit payload in DVWA SQL Injection module

The commands or workflow above represent the evidence-producing action. Placeholders such as , , or should be used in public documentation instead of live secrets or private infrastructure details.

4. Professional Interpretation

The screenshot shows the injected payload returning multiple user records, demonstrating SQL Injection behaviour in the lab.

This evidence should be treated as a supporting artifact, not as a standalone finding by itself. In a professional report, the screenshot supports a specific testing phase, validates an observation, and strengthens reproducibility when paired with methodology and remediation guidance.

5. Security Risk / Why It Matters

In production, SQLi can enable authentication bypass, data extraction, modification, and full database compromise.

For this portfolio project, the risk is described in a real-world context while clearly noting that the target was an authorized DVWA/local lab environment. This avoids exaggerating lab results as production impact.

6. Recommended Remediation / Defensive Control

Use parameterized queries, prepared statements, input validation, least-privilege database accounts, and secure error handling.

7. GitHub Redaction Requirements

User records, names, database output, cookies, session IDs, and any credentials.

Before pushing to GitHub, confirm that screenshots do not expose cookies, session IDs, credentials, hashes, cracked passwords, private IPs, internal file paths, or any secret values. If the screenshot already contains sensitive information, blur or crop it before upload.

8. GitHub README Link Snippet

```
[Open Evidence PDF](./reports/evidence-file-documentation-with-screenshots/14_09-dvwa-sqli-vulnerable-input-response_evidence_documentation.pdf)
```

Legal and Ethical Note

This evidence documents authorized testing against an intentionally vulnerable local lab target. The techniques described must not be used against public or third-party systems without explicit permission.